RECONCILEPRO CORE ENGINE • CODE TECHNICAL SPECIFICATION

reconcile.py — GST Reconciliation Engine

A comprehensive functional walkthrough of the automated reconciliation logic, schema normalization, mathematical set partitioning, and statutory discrepancy classification.

TARGET FILE	MODULE PATH	PYTHON VERSION	DOMAIN
reconcile.py	backend/reconciliation	Python 3.10+ (Pandas)	GST MSME Compliance

1 Executive Summary & Architectural Role

The module `backend/reconciliation/reconcile.py` represents the computational nucleus of the **ReconcilePro** compliance platform. Under Indian Goods and Services Tax regulations, registered taxpayers must reconcile their internal accounting ledgers against government portal filings on a periodic basis to claim **Input Tax Credit (ITC)** and avoid statutory penalties under Section 16(2)(aa) and Section 50 of the CGST Act.

Outward Reconciliation (Sales)

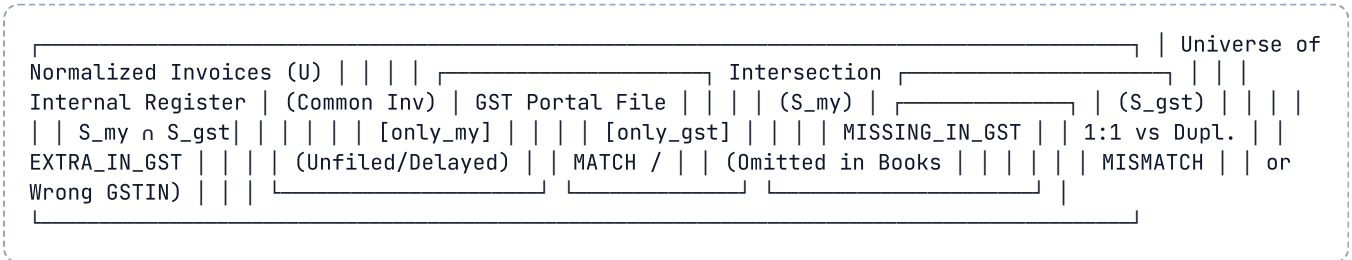
Compares internal **Sales Register** against filed **GSTR-1**. Identifies unfiled customer invoices (underreported tax liability) or duplicate tax bookings.

Inward Reconciliation (Purchases)

Compares internal **Purchase Register** against counterparty auto-populated **GSTR-2A / GSTR-2B** returns. Protects ineligible ITC claims.

Mathematical Engine Architecture

The algorithm enforces a 3-way disjoint set partition across the universe of invoice identifiers:



Classification Taxonomy

Status Category	Condition / Trigger	Compliance Implication & Impact
MATCH	`tv_diff == 0 and gst_diff == 0`	Invoice verified. Values coincide precisely between books and GST portal. Zero audit risk.
MISMATCH	`tv_diff ≠ 0 or gst_diff ≠ 0`	Taxable value or GST calculation discrepancy. Suggests data entry typos, unrecorded credit notes, or rate discrepancies.

Status Category	Condition / Trigger	Compliance Implication & Impact
DUPLICATE	`n_my > 1 or n_gst > 1`	Duplicate invoice numbers recorded. Multi-row preserve algorithm isolates each duplicate to prevent silent data drop.
MISSING_IN_GST	`inv in S_my - S_gst`	Present in books, absent on portal. For sales: unfiled return. For purchases: vendor failed to file GSTR-1, blocking ITC.
EXTRA_IN_GST	`inv in S_gst - S_my`	Present on portal, missing in books. Possible unrecorded purchase, wrong recipient GSTIN, or timing difference.

2 Function 1: `_extract_gstin(row)`

A helper function designed to extract and normalize the 15-character Goods and Services Taxpayer Identification Number (GSTIN) from heterogeneous tabular records.

```
def _extract_gstin(row) -> str:
    if hasattr(row, "get"):
        return str(row.get("gstin") or row.get("customer_gstin") or row.get("supplier_gstin") or
"").strip()
    return ""
```

Technical Deep Dive & Design Rationale

1. **Polymorphic Type Guarding:** Evaluates `hasattr(row, "get")` to seamlessly accept either a Python `dict` or a Pandas `Series` (both implement `.get()`). Safely returns an empty string if passed primitives or nulls.
2. **Short-Circuit Priority Resolution:** Uses Python's boolean operator chain (`or`) to evaluate candidate columns in priority order:
   - `gstin` : Generic standard column name.
   - `customer_gstin` : Typical outward supply identifier (Sales Register, GSTR-1 B2B).
   - `supplier_gstin` : Typical inward supply identifier (Purchase Register, GSTR-2A/2B).
3. **Sanitization & Defensive Formatting:** Casts the extracted object to `str` and executes `.strip()` to eliminate trailing spaces, carriage returns, or newline artifacts imported from CSV/Excel files.

3 Function 2: `reconcile(my_source, gst_source)` — Core Engine

The primary reconciliation orchestrator. Accepts two heterogeneous data sources (internal books vs GST filings) and outputs a unified discrepancy audit DataFrame.

Phase 1 & 2: Source Ingestion and Defensive Schema Initialization

```
def reconcile(my_source, gst_source):
    # Phase 1: Dual Input Support (Pandas DataFrame or File Path)
    if isinstance(my_source, pd.DataFrame):
        my_df = my_source.copy()
    else:
        my_path = my_source if isinstance(my_source, Path) else (DATA / my_source)
        my_df = pd.read_csv(my_path)

    if isinstance(gst_source, pd.DataFrame):
        gst_df = gst_source.copy()
    else:
        gst_path = gst_source if isinstance(gst_source, Path) else (DATA / gst_source)
        gst_df = pd.read_csv(gst_path)

    # Phase 2: Schema Enforcement & Defaulting
    for col in ["invoice_no", "invoice_date", "gstin", "taxable_value", "gst_amount"]:
        if col not in my_df.columns:
            my_df[col] = pd.Series(dtype=object)
```

```
        if col not in gst_df.columns:
            gst_df[col] = pd.Series(dtype=object)

    my_df["invoice_no"] = my_df["invoice_no"].astype(str).str.strip()
    gst_df["invoice_no"] = gst_df["invoice_no"].astype(str).str.strip()

    # Filter empty placeholder / blank rows
    my_df = my_df[my_df["invoice_no"] != ""].copy() if not my_df.empty else my_df
    gst_df = gst_df[gst_df["invoice_no"] != ""].copy() if not gst_df.empty else gst_df
```

Critical Architectural Highlights in Phase 1 & 2

- **In-Memory Copy Safety:** Using `.copy()` guarantees functional purity; caller DataFrames (such as those passed from FastAPI route handlers) are not mutated by column injection or trimming.
- **Polymorphic Ingestion:** Supports both CLI offline testing (loading CSVs from disk via `DATA / filename`) and web application execution (passing filtered database query results directly as DataFrames).
- **Defensive Column Imputation:** Proactively injects mandatory columns if omitted in user uploads, preventing downstream `KeyError` application crashes.
- **String Coercion:** Converts numeric invoice numbers (e.g., `1001` parsed as float or integer) into clean stripped strings to avoid type-mismatch hash collisions in set operations.

4 Mathematical Set Partitioning & 1:1 Matching

The engine projects the cleaned invoice identifiers into Python hash sets, achieving theoretical $O(N + M)$ performance for discrepancy identification.

```
    my_inv = set(my_df["invoice_no"]) if not my_df.empty else set()
    gst_inv = set(gst_df["invoice_no"]) if not gst_df.empty else set()

    common = my_inv & gst_inv      # S_my ∩ S_gst: Invoices present in both
    only_my = my_inv - gst_inv     # S_my \ S_gst: Present in books, absent on portal
    only_gst = gst_inv - my_inv    # S_gst \ S_my: Present on portal, absent in books
```

1-to-1 Match & Mathematical Tolerance Verification

When an invoice appears exactly once in the internal books (`n_my == 1`) and exactly once on the GST portal (`n_gst == 1`), the engine performs arithmetic difference verification:

```
    for inv in sorted(common):
        my_matches = my_df[my_df["invoice_no"] == inv]
        gst_matches = gst_df[gst_df["invoice_no"] == inv]
        n_my = len(my_matches)
        n_gst = len(gst_matches)

        if n_my == 1 and n_gst == 1:
            a = my_matches.iloc[0]
            b = gst_matches.iloc[0]

            a_tv = float(a["taxable_value"])
            b_tv = float(b["taxable_value"])
            a_gst = float(a["gst_amount"])
            b_gst = float(b["gst_amount"])

            tv_diff = round(a_tv - b_tv, 2)
            gst_diff = round(a_gst - b_gst, 2)

            if tv_diff == 0 and gst_diff == 0:
                status = "MATCH"
                explanation = "Invoice matched across both internal records and GST portal."
                category = "MATCH"
                possible_causes = []
                action = "No action required. Reconciled successfully."
            else:
                status = "MISMATCH"
                diff_reasons = []
                if tv_diff ≠ 0:
                    diff_reasons.append(f"Taxable value differs by Rs. {abs(tv_diff):,.2f}")
                if gst_diff ≠ 0:
                    diff_reasons.append(f"GST amount differs by Rs. {abs(gst_diff):,.2f}")
                explanation = ". ".join(diff_reasons) + " between internal books and GST filing."
                category = ("TAXABLE_AND_GST_MISMATCH" if (tv_diff ≠ 0 and gst_diff ≠ 0)
                            else ("TAXABLE_VALUE_MISMATCH" if tv_diff ≠ 0 else
"GST_AMOUNT_MISMATCH"))
                possible_causes = [
                    "Hypothesis: Data entry discrepancy in taxable value or rate in internal
ledger.",
                    "Hypothesis: Commercial discount or credit/debit note applied on one side
only.",
                    "Hypothesis: Rounding difference across line items.",
```

```
                ]
                action = "Compare physical invoice copy against filed portal return. Request
amendment or credit/debit note if needed."
```

Root Cause Hypotheses & Actionable Remediation

Rather than simply returning a boolean flag, the engine populates structured diagnostic metadata (`explanation` , `discrepancy_category` , `possible_causes` , `suggested_action`). This enables accounting clerks to immediately know what steps to take (e.g., verifying physical copies, issuing credit notes, or updating internal rates).

5 Safe Duplicate Resolution Algorithm

In traditional naive reconciliation algorithms, a common bug occurs when an invoice number appears more than once: calling `.iloc[0]` silently discards subsequent rows, hiding duplicate billings and causing serious compliance exposure. `reconcile.py` implements a comprehensive zero-loss duplicate pair algorithm.

```
        else:
            # Safe duplicate handling: preserve every row without silent .iloc[0] drop
            max_len = max(n_my, n_gst)
            for idx in range(max_len):
                a = my_matches.iloc[idx] if idx < n_my else None
                b = gst_matches.iloc[idx] if idx < n_gst else None

                a_tv = float(a["taxable_value"]) if a is not None else None
                b_tv = float(b["taxable_value"]) if b is not None else None
                a_gst = float(a["gst_amount"]) if a is not None else None
                b_gst = float(b["gst_amount"]) if b is not None else None

                tv_diff = round(a_tv - b_tv, 2) if (a_tv is not None and b_tv is not None) else None
                gst_diff = round(a_gst - b_gst, 2) if (a_gst is not None and b_gst is not None) else
None

                explanation = f"Duplicate invoice number '{inv}' detected ({n_my} in books, {n_gst}
in GST). Requires manual review."
                category = "DUPLICATE_INVOICE"
```

6 Partition Asymmetries: MISSING_IN_GST & EXTRA_IN_GST

MISSING_IN_GST (only_my)

Detection: `inv in (my_inv - gst_inv)`
Sales Context: Invoice recorded in enterprise sales register was not uploaded to GSTR-1. Leads to tax underpayment penalties under Section 50.
Purchase Context: Vendor failed to file their GSTR-1 before statutory cut-off. **ITC is blocked** under Section 16(2)(aa).
Remediation: Push invoice to next GSTR-1 period or follow up with vendor.

EXTRA_IN_GST (only_gst)

Detection: `inv in (gst_inv - my_inv)`
Sales Context: Invoice exists on portal but missing in ERP. Possible unrecorded cash sale or wrong GSTIN entered during return submission.
Purchase Context: Vendor uploaded an invoice that the business has not booked. May represent goods in transit, unbooked bill, or wrong buyer GSTIN.
Remediation: Inspect delivery challans; enter in purchase journal or request counterparty cancel wrong entry.

7 Function 3: `show_result(title, df)`

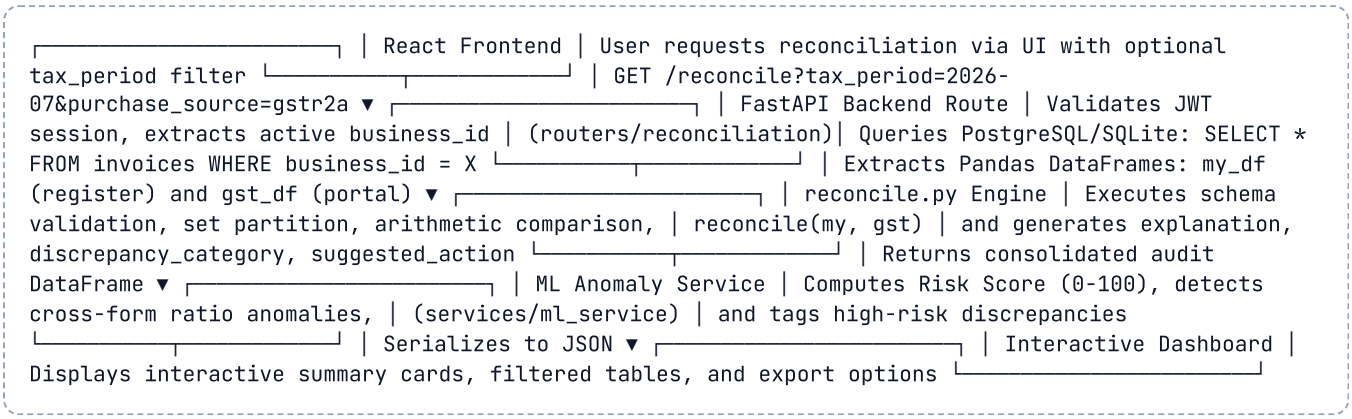
Provides terminal formatting and summary telemetry for standalone execution:

```
def show_result(title, df):
    print("
" + "=" * 60)
    print(title)
    print("=" * 60)
```

```
    print(df.to_string(index=False))
    print("
Summary:")
    print(df["status"].value_counts())
```

8 End-to-End System Integration Flow

How `reconcile.py` integrates across the ReconcilePro architecture:



9 Output Schema Reference Table

Column Name	Data Type	Description / Purpose
`invoice_no`	String	Normalized, trimmed invoice identifier string.
`invoice_date`	String (YYYY-MM-DD)	Date of document issuance.
`gstin`	String (15-char)	Counterparty GSTIN extracted via `_extract_gstin`.
`expected_taxable_value`	Float / Null	Taxable value recorded in internal accounts register.
`actual_taxable_value`	Float / Null	Taxable value reported on government GST portal return.
`taxable_value_diff`	Float / Null	Arithmetic variance: `round(expected - actual, 2)`.
`expected_gst_amount`	Float / Null	GST amount (CGST + SGST + IGST) recorded in internal accounts.
`actual_gst_amount`	Float / Null	GST amount reported on GST portal return.
`gst_amount_diff`	Float / Null	Arithmetic variance: `round(expected_gst - actual_gst, 2)`.
`status`	Enum String	Primary classification: `MATCH`, `MISMATCH`, `MISSING_IN_GST`, `EXTRA_IN_GST`.
`discrepancy_category`	Enum String	Specific taxonomy (e.g., `TAXABLE_AND_GST_MISMATCH`, `DUPLICATE_INVOICE`).
`explanation`	String	Human-readable audit sentence detailing variances.
`possible_causes`	List[String]	Plausible bookkeeping or filing hypotheses.
`suggested_action`	String	Statutory compliance next step for the finance team.